

Sticky Note Roulette — Complete Implementation Guide

- ⋮ A prototype that connects Miro's visual collaboration with Qwen's AI creativity. Add random sticky notes to a Miro board, press a button, and watch AI find impossible connections.

Project Structure

```
sticky-note-roulette/  
├─ package.json  
├─ .env.example  
├─ server.js  
├─ public/  
│   ├─ index.html  
│   ├─ style.css  
│   └─ app.js  
└─ README.md
```

File: package.json

```
{  
  "name": "sticky-note-roulette",  
  "version": "1.0.0",  
  "description": "AI-powered creative connections for Miro sticky notes",  
  "main": "server.js",  
  "scripts": {  
    "start": "node server.js",  
    "dev": "node server.js"  
  },  
  "dependencies": {  
    "express": "^4.18.2",  
    "cors": "^2.8.5",  
    "dotenv": "^16.3.1",  
    "node-fetch": "^2.7.0",  
    "@mirohq/miro-api": "^1.4.0"  
  },  
  "engines": {  
    "node": ">=16.0.0"  
  }  
}
```

File: .env.example

```
MIRO_ACCESS_TOKEN=your_miro_access_token  
MIRO_BOARD_ID=your_board_id  
QWEN_API_KEY=your_qwen_api_key  
QWEN_API_URL=https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions  
QWEN_MODEL=qwen-plus  
PORT=3000
```

File: server.js

```

require('dotenv').config();
const express = require('express');
const cors = require('cors');
const fetch = require('node-fetch');
const path = require('path');

const app = express();
const PORT = process.env.PORT || 3000;

app.use(cors());
app.use(express.json());
app.use(express.static('public'));

const MIRO_BASE_URL = 'https://api.miro.com/v2';
const MIRO_TOKEN = process.env.MIRO_ACCESS_TOKEN;
const MIRO_BOARD_ID = process.env.MIRO_BOARD_ID;
const QWEN_API_URL = process.env.QWEN_API_URL;
const QWEN_API_KEY = process.env.QWEN_API_KEY;
const QWEN_MODEL = process.env.QWEN_MODEL || 'qwen-plus';

app.get('/api/sticky-notes', async (req, res) => {
  try {
    const response = await fetch(
      `${MIRO_BASE_URL}/boards/${MIRO_BOARD_ID}/items?type=sticky_note&limit=100`,
      {
        headers: {
          'Authorization': `Bearer ${MIRO_TOKEN}`,
          'Content-Type': 'application/json'
        }
      }
    );

    if (!response.ok) {
      throw new Error(`Miro API error: ${response.status}`);
    }

    const data = await response.json();
    const stickyNotes = data.data.map(note => ({
      id: note.id,
      text: note.data.content.replace(/<[^>]*>/g, '').trim(),
      position: note.position
    }));

    res.json(stickyNotes);
  } catch (error) {
    console.error('Error fetching sticky notes:', error);
    res.status(500).json({ error: 'Failed to fetch sticky notes from Miro' });
  }
});

app.post('/api/connect', async (req, res) => {
  try {
    const { notes } = req.body;

    if (!notes || notes.length < 2) {
      return res.status(400).json({ error: 'Need at least 2 sticky notes to connect' });
    }

    const notesText = notes.map(n => n.text).join(', ');
  }
});

```

```
const prompt = `You are an expert creative ideation engine.
```

You will receive a collection of unrelated sticky notes from a Miro board.

Find surprising but genuinely useful connections between them.

Rules:

- Do not simply combine the words literally.
- Look for conceptual relationships.
- Prefer ideas that could actually be built.
- Each idea must use 2-4 source notes.
- Generate exactly 3 ideas.
- Make each idea substantially different.
- One idea should be practical.
- One should be unusual.
- One should be borderline ridiculous but still logically defensible.
- Avoid generic startup ideas.
- Explain why the selected notes connect.
- Return only valid JSON.

The sticky notes are: \${notesText}

Return this exact JSON structure:

```
{
  "ideas": [
    {
      "title": "string",
      "description": "string",
      "source_notes": ["string", "string"],
      "connection": "string",
      "buildability": 8
    }
  ]
}
```

```
const qwenResponse = await fetch(QWEN_API_URL, {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${QWEN_API_KEY}`,
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    model: QWEN_MODEL,
    messages: [
      {
        role: 'system',
        content: 'You are a creative ideation assistant. Always respond with valid JSON.'
      },
      {
        role: 'user',
        content: prompt
      }
    ],
    temperature: 0.9
  })
});
```

```
if (!qwenResponse.ok) {
  throw new Error(`Qwen API error: ${qwenResponse.status}`);
}
```

```

    }

    const qwenData = await qwenResponse.json();
    const content = qwenData.choices[0].message.content;

    let ideas;
    try {
        const jsonMatch = content.match(/\{[\s\S]*\}/);
        if (jsonMatch) {
            ideas = JSON.parse(jsonMatch[0]);
        } else {
            ideas = JSON.parse(content);
        }
    } catch (parseError) {
        console.error('JSON parse error:', content);
        throw new Error('Failed to parse Qwen response as JSON');
    }

    res.json(ideas);
} catch (error) {
    console.error('Error connecting ideas:', error);
    res.status(500).json({ error: 'Failed to generate connections with Qwen' });
}
});

app.post('/api/create-ideas', async (req, res) => {
    try {
        const { ideas, sourceNotes } = req.body;

        const baseX = 1500;
        const baseY = 0;
        const spacing = 400;

        const createdIdeas = [];

        for (let i = 0; i < ideas.length; i++) {
            const idea = ideas[i];

            const contentHtml = `<p><strong>${idea.title}</strong></p><p>${idea.description}</p><p><em>Inspired by: ${idea.source_notes.join(', ')}</em></p><p>Why: ${idea.connection}</p><p>Buildability: ${idea.buildability}/10</p>`;

            const shapeResponse = await fetch(
                `${MIRO_BASE_URL}/boards/${MIRO_BOARD_ID}/shapes`,
                {
                    method: 'POST',
                    headers: {
                        'Authorization': `Bearer ${MIRO_TOKEN}`,
                        'Content-Type': 'application/json'
                    },
                    body: JSON.stringify({
                        data: {
                            content: contentHtml,
                            shape: 'round_rectangle'
                        },
                        style: {
                            fillColor: '#1a1a2e',
                            fontFamily: 'open_sans',
                            fontSize: '14',
                            textAlign: 'left',

```

```

        color: '#ffffff',
        borderColor: '#7c3aed',
        borderWidth: '2.0'
      },
      position: {
        x: baseX,
        y: baseY + (i * spacing)
      },
      geometry: {
        width: 400,
        height: 300
      }
    })
  }
}
);

if (!shapeResponse.ok) {
  throw new Error(`Failed to create shape: ${shapeResponse.status}`);
}

const shapeData = await shapeResponse.json();
const shapeId = shapeData.id;

for (const sourceText of idea.source_notes) {
  const matchingNote = sourceNotes.find(n => n.text === sourceText);
  if (matchingNote) {
    await fetch(
      `${MIRO_BASE_URL}/boards/${MIRO_BOARD_ID}/connectors`,
      {
        method: 'POST',
        headers: {
          'Authorization': `Bearer ${MIRO_TOKEN}`,
          'Content-Type': 'application/json'
        },
        body: JSON.stringify({
          startItem: { id: matchingNote.id },
          endItem: { id: shapeId },
          style: {
            strokeColor: '#7c3aed',
            strokeWidth: '2.0'
          }
        })
      })
  }
}
);
}
}

createdIdeas.push({ ...idea, shapeId });
}

res.json({ success: true, ideas: createdIdeas });
} catch (error) {
  console.error('Error creating ideas on board:', error);
  res.status(500).json({ error: 'Failed to create ideas on Miro board' });
}
});

app.post('/api/make-weirder', async (req, res) => {
  try {
    const { ideas, sourceNotes } = req.body;

```

```

const ideasSummary = ideas.map(i => `${i.title}: ${i.description}`).join('\n');

const prompt = `You previously generated these creative ideas from sticky notes:

${ideasSummary}

Push each idea further. Make them weirder, more ambitious, more unexpected – but still
logically defensible. Preserve the core connection but amplify the creativity to 11.

Return the same JSON structure with 3 evolved ideas:
{
  "ideas": [
    {
      "title": "string",
      "description": "string",
      "source_notes": ["string", "string"],
      "connection": "string",
      "buildability": 8
    }
  ]
};

const qwenResponse = await fetch(QWEN_API_URL, {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${QWEN_API_KEY}`,
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    model: QWEN_MODEL,
    messages: [
      {
        role: 'system',
        content: 'You are a creative ideation assistant. Push ideas to their
        creative limits. Always respond with valid JSON.'
      },
      {
        role: 'user',
        content: prompt
      }
    ],
    temperature: 1.0
  })
});

if (!qwenResponse.ok) {
  throw new Error(`Qwen API error: ${qwenResponse.status}`);
}

const qwenData = await qwenResponse.json();
const content = qwenData.choices[0].message.content;

let weirderIdeas;
try {
  const jsonMatch = content.match(/\{[\s\S]*\}/);
  if (jsonMatch) {
    weirderIdeas = JSON.parse(jsonMatch[0]);
  } else {
    weirderIdeas = JSON.parse(content);
  }
}

```

```

} catch (parseError) {
  console.error('JSON parse error:', content);
  throw new Error('Failed to parse Qwen response as JSON');
}

const baseX = 2100;
const baseY = 0;
const spacing = 400;

const createdIdeas = [];

for (let i = 0; i < weirderIdeas.ideas.length; i++) {
  const idea = weirderIdeas.ideas[i];

  const contentHtml = `<p><strong>🌀 ${idea.title}</strong></p><p>${idea
    .description}</p><p><em>Inspired by: ${idea.source_notes.join(', ')}</em></p>
    <p>Why: ${idea.connection}</p><p>Buildability: ${idea.buildability}/10</p>`;

  const shapeResponse = await fetch(
    `${MIRO_BASE_URL}/boards/${MIRO_BOARD_ID}/shapes`,
    {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${MIRO_TOKEN}`,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        data: {
          content: contentHtml,
          shape: 'round_rectangle'
        },
        style: {
          fillColor: '#2e1a1a',
          fontFamily: 'open_sans',
          fontSize: '14',
          textAlign: 'left',
          color: 'ffffff',
          borderColor: '#ff6b35',
          borderWidth: '3.0'
        },
        position: {
          x: baseX,
          y: baseY + (i * spacing)
        },
        geometry: {
          width: 400,
          height: 300
        }
      })
    }
  );

  if (!shapeResponse.ok) {
    throw new Error(`Failed to create shape: ${shapeResponse.status}`);
  }

  const shapeData = await shapeResponse.json();
  createdIdeas.push({ ...idea, shapeId: shapeData.id });
}

res.json({ ideas: weirderIdeas.ideas, created: createdIdeas });

```

```
    } catch (error) {  
      console.error('Error making ideas weirder:', error);  
      res.status(500).json({ error: 'Failed to amplify ideas with Qwen' });  
    }  
  });  
  
app.listen(PORT, () => {  
  console.log(`📝 Sticky Note Roulette running on http://localhost:${PORT}`);  
});
```

File: public/index.html

```

<!DOCTYPE html>
<html lang=&quot;en&quot;>
<head>
  <meta charset=&quot;UTF-8&quot;>
  <meta name=&quot;viewport&quot; content=&quot;width=device-width, initial-scale=1
    .0&quot;>
  <title>Sticky Note Roulette</title>
  <link rel=&quot;stylesheet&quot; href=&quot;style.css&quot;>
</head>
<body>
  <div class=&quot;container&quot;>
    <header class=&quot;hero&quot;>
      <h1 class=&quot;title&quot;>📝 STICKY NOTE ROULETTE</h1>
      <p class=&quot;subtitle&quot;>Random thoughts → Miro → AI → impossible connections
        </p>
    </header>

    <div class=&quot;found-notes&quot; id=&quot;foundNotes&quot;>
      <p class=&quot;found-label&quot;>Found on your board:</p>
      <div class=&quot;notes-tags&quot; id=&quot;notesTags&quot;></div>
    </div>

    <div class=&quot;actions&quot;>
      <button class=&quot;btn-primary&quot; id=&quot;connectBtn&quot;>
        <span class=&quot;btn-text&quot;>CONNECT THE IMPOSSIBLE</span>
        <span class=&quot;btn-loader&quot; style=&quot;display: none;&quot;>●●●</span>
      </button>
    </div>

    <div class=&quot;error-message&quot; id=&quot;errorMessage&quot;></div>

    <div class=&quot;ideas-container&quot; id=&quot;ideasContainer&quot; style=&quot;
      ;display: none;&quot;>
      <h2 class=&quot;ideas-title&quot;>Generated Connections</h2>
      <div class=&quot;ideas-grid&quot; id=&quot;ideasGrid&quot;></div>

      <div class=&quot;actions&quot; style=&quot;margin-top: 2rem;&quot;>
        <button class=&quot;btn-weirder&quot; id=&quot;weirderBtn&quot;>
          <span>🌀 MAKE IT WEIRDER</span>
        </button>
      </div>
    </div>

    <div class=&quot;weirder-container&quot; id=&quot;weirderContainer&quot; style=&quot;
      ;display: none;&quot;>
      <h2 class=&quot;ideas-title&quot;>Amplified Weirderness</h2>
      <div class=&quot;ideas-grid&quot; id=&quot;weirderGrid&quot;></div>
    </div>
  </div>

  <script src=&quot;app.js&quot;></script>
</body>
</html>

```

File: public/style.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto,
    Helvetica, Arial, sans-serif;
  background: #0a0a0a;
  color: #f0f0f0;
  line-height: 1.6;
  min-height: 100vh;
}

.container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 3rem 2rem;
}

.hero {
  text-align: center;
  margin-bottom: 3rem;
}

.title {
  font-size: 4rem;
  font-weight: 900;
  letter-spacing: -0.02em;
  background: linear-gradient(135deg, #7c3aed 0%, #ec4899 100%);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
  margin-bottom: 1rem;
}

.subtitle {
  font-size: 1.25rem;
  color: #9ca3af;
  font-weight: 300;
}

.found-notes {
  background: #1a1a1a;
  border: 1px solid #2a2a2a;
  border-radius: 12px;
  padding: 1.5rem;
  margin-bottom: 2rem;
}

.found-label {
  font-size: 0.875rem;
  color: #9ca3af;
  margin-bottom: 1rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
}
```

```
.notes-tags {
  display: flex;
  flex-wrap: wrap;
  gap: 0.75rem;
}

.note-tag {
  background: #7c3aed;
  color: #ffffff;
  padding: 0.5rem 1rem;
  border-radius: 20px;
  font-size: 0.875rem;
  font-weight: 500;
}

.actions {
  text-align: center;
  margin: 2rem 0;
}

.btn-primary {
  background: #7c3aed;
  color: #ffffff;
  border: 2px solid #7c3aed;
  padding: 1.5rem 3rem;
  font-size: 1.5rem;
  font-weight: 700;
  border-radius: 12px;
  cursor: pointer;
  transition: all 0.3s ease;
  box-shadow: 0 0 20px rgba(124, 58, 237, 0.4);
  text-transform: uppercase;
  letter-spacing: 0.05em;
}

.btn-primary:hover {
  background: #6d28d9;
  box-shadow: 0 0 30px rgba(124, 58, 237, 0.6);
  transform: translateY(-2px);
}

.btn-primary:active {
  transform: translateY(0);
}

.btn-primary.loading {
  background: #4c1d95;
  pointer-events: none;
}

.btn-loader {
  animation: pulse 1.5s infinite;
}

@keyframes pulse {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.3; }
}

.btn-weirder {
  background: #ff6b35;
}
```

```
color: #ffffff;
border: 2px solid #ff6b35;
padding: 1.25rem 2.5rem;
font-size: 1.25rem;
font-weight: 700;
border-radius: 12px;
cursor: pointer;
transition: all 0.3s ease;
box-shadow: 0 0 20px rgba(255, 107, 53, 0.4);
text-transform: uppercase;
letter-spacing: 0.05em;
}

.btn-weirder:hover {
  background: #e85d2e;
  box-shadow: 0 0 30px rgba(255, 107, 53, 0.6);
  transform: translateY(-2px);
}

.error-message {
  background: #991b1b;
  color: #fecaca;
  padding: 1rem 1.5rem;
  border-radius: 8px;
  margin: 1rem 0;
  display: none;
  text-align: center;
  border: 1px solid #dc2626;
}

.error-message.show {
  display: block;
}

.ideas-container,
.weirder-container {
  margin-top: 3rem;
}

.ideas-title {
  font-size: 2rem;
  font-weight: 700;
  margin-bottom: 2rem;
  text-align: center;
  color: #f0f0f0;
}

.ideas-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(350px, 1fr));
  gap: 2rem;
  margin-bottom: 2rem;
}

.idea-card {
  background: #1a1a1a;
  border-left: 4px solid #7c3aed;
  border-radius: 8px;
  padding: 2rem;
  transition: transform 0.2s ease;
}
```

```
}

.idea-card:hover {
  transform: translateY(-4px);
}

.idea-card.weirder {
  border-left-color: #ff6b35;
}

.idea-title {
  font-size: 1.5rem;
  font-weight: 700;
  color: #ffffff;
  margin-bottom: 1rem;
}

.idea-description {
  font-size: 1rem;
  color: #d1d5db;
  margin-bottom: 1.5rem;
  line-height: 1.7;
}

.idea-meta {
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}

.idea-sources {
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem;
}

.source-pill {
  background: #374151;
  color: #9ca3af;
  padding: 0.25rem 0.75rem;
  border-radius: 12px;
  font-size: 0.75rem;
  font-weight: 500;
}

.idea-connection {
  color: #9ca3af;
  font-size: 0.875rem;
  font-style: italic;
  padding-left: 0.5rem;
  border-left: 2px solid #374151;
}

.idea-buildability {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  margin-top: 1rem;
}

.buildability-label {
```

```
font-size: 0.875rem;
color: #9ca3af;
font-weight: 600;
}

.buildability-bar {
  flex: 1;
  height: 8px;
  background: #374151;
  border-radius: 4px;
  overflow: hidden;
}

.buildability-fill {
  height: 100%;
  background: linear-gradient(90deg, #7c3aed 0%, #ec4899 100%);
  border-radius: 4px;
  transition: width 0.5s ease;
}

@media (max-width: 768px) {
  .title {
    font-size: 2.5rem;
  }

  .subtitle {
    font-size: 1rem;
  }

  .btn-primary {
    font-size: 1.25rem;
    padding: 1.25rem 2rem;
  }

  .ideas-grid {
    grid-template-columns: 1fr;
  }
}
```

File: public/app.js

```

let stickyNotes = [];
let currentIdeas = [];

async function loadStickyNotes() {
  try {
    const response = await fetch(`${API}/api/sticky-notes`);
    if (!response.ok) {
      throw new Error('Failed to fetch sticky notes');
    }

    const notes = await response.json();
    stickyNotes = notes;

    const notesTags = document.getElementById('notesTags');
    notesTags.innerHTML = '';

    if (notes.length === 0) {
      notesTags.innerHTML = `<p style="color: #9ca3af;">No sticky notes found. Add some to your Miro board first!</p>`;
      document.getElementById('connectBtn').disabled = true;
      return;
    }

    notes.forEach(note => {
      const tag = document.createElement('span');
      tag.className = 'note-tag';
      tag.textContent = note.text;
      notesTags.appendChild(tag);
    });

  } catch (error) {
    console.error('Error loading sticky notes:', error);
    showError('Failed to load sticky notes from Miro. Check your connection and credentials.');
```

```

        btn.textContent = '<span>Processing...</span>';
    }
} else {
    btn.classList.remove('<span>loading</span>');
    btn.disabled = false;
    if (text) text.style.display = '<span>inline</span>';
    if (loader) {
        loader.style.display = '<span>none</span>';
    } else if (btn.dataset.originalText) {
        btn.textContent = btn.dataset.originalText;
    }
}
}
}

function renderIdea(idea, isWeirder = false) {
    const card = document.createElement('<div>');
    card.className = `idea-card ${isWeirder ? '<span>weirder</span>' : '<span></span>'}`;

    const sourcePills = idea.source_notes.map(note =>
        '<span class="source-pill">${note}</span>'
    ).join('<span></span>');

    const buildabilityPercent = (idea.buildability / 10) * 100;

    card.innerHTML = `
        <h3 class="idea-title">${isWeirder ? '<span>🌀 </span>' : '<span></span>'}${idea
            .title}</h3>
        <p class="idea-description">${idea.description}</p>
        <div class="idea-meta">
            <div class="idea-sources">${sourcePills}</div>
            <p class="idea-connection">${idea.connection}</p>
            <div class="idea-buildability">
                <span class="buildability-label">Buildability</span>
                <div class="buildability-bar">
                    <div class="buildability-fill" style="width:
                        ${buildabilityPercent}%"></div>
                </div>
                <span class="buildability-label">${idea.buildability}/10</span>
            </div>
        </div>
    `;

    return card;
}

async function connectIdeas() {
    if (stickyNotes.length < 2) {
        showError('<span>Give me at least 2 sticky notes to connect.</span>');
        return;
    }

    const connectBtn = document.getElementById('<span>connectBtn</span>');
    setLoading(connectBtn, true);

    try {
        const response = await fetch('<span>/api/connect</span>', {
            method: '<span>POST</span>',
            headers: { '<span>Content-Type</span>': '<span>application/json</span>' },
            body: JSON.stringify({ notes: stickyNotes })
        });
    }
}

```

```

    if (!response.ok) {
      throw new Error(`${Failed to connect ideas}`);
    }

    const data = await response.json();
    currentIdeas = data.ideas;

    const ideasGrid = document.getElementById(`${ideasGrid}`);
    ideasGrid.innerHTML = ``;

    currentIdeas.forEach(idea => {
      ideasGrid.appendChild(renderIdea(idea));
    });

    document.getElementById(`${ideasContainer}`).style.display = `block`;

    await fetch(`${api/create-ideas}`, {
      method: `POST`,
      headers: { `Content-Type`: `application/json` },
      body: JSON.stringify({
        ideas: currentIdeas,
        sourceNotes: stickyNotes
      })
    });

  } catch (error) {
    console.error(`${Error connecting ideas}`, error);
    showError(`${Something went wrong connecting to Qwen. Try again?}`);
  } finally {
    setLoading(connectBtn, false);
  }
}

async function makeWeirder() {
  const weirderBtn = document.getElementById(`${weirderBtn}`);
  setLoading(weirderBtn, true);

  try {
    const response = await fetch(`${api/make-weirder}`, {
      method: `POST`,
      headers: { `Content-Type`: `application/json` },
      body: JSON.stringify({
        ideas: currentIdeas,
        sourceNotes: stickyNotes
      })
    });

    if (!response.ok) {
      throw new Error(`${Failed to amplify ideas}`);
    }

    const data = await response.json();

    const weirderGrid = document.getElementById(`${weirderGrid}`);
    weirderGrid.innerHTML = ``;

    data.ideas.forEach(idea => {
      weirderGrid.appendChild(renderIdea(idea, true));
    });
  }
}

```

```

    }},

    document.getElementById(&apos;weirderContainer&apos;).style.display = &apos;
      ;block&apos;;

  } catch (error) {
    console.error(&apos;Error making ideas weirder:&apos;;, error);
    showError(&apos;Failed to amplify weirdness. Try again?&apos;);
  } finally {
    setLoading(weirderBtn, false);
  }
}

document.getElementById(&apos;connectBtn&apos;).addEventListener(&apos;click&apos;;,
  connectIdeas);
document.getElementById(&apos;weirderBtn&apos;).addEventListener(&apos;click&apos;;,
  makeWeirder);

loadStickyNotes();

```

File: README.md

```

# Sticky Note Roulette 🎰

Random thoughts → Miro → Qwen → unexpected connections → new ideas on your board.

A prototype that connects Miro's visual collaboration with Qwen's AI
creativity. Add random sticky notes to a Miro board, press a button, and watch AI find
impossible connections between unrelated ideas.

---

## Setup (5 minutes)

### 1. Clone and Install



```

`bash
git clone <your-repo-url>
cd sticky-note-roulette
npm install

```


```

2. Get Your API Keys

Miro Access Token:

1. Go to <https://miro.com/app/settings/user-profile/apps>
2. Create a new app
3. Generate an access token with these scopes:
 - `boards:read`
 - `boards:write`

4. Copy the token

Miro Board ID:

1. Open your Miro board
2. The board ID is in the URL: `https://miro.com/app/board/[THIS-IS-YOUR-BOARD-ID]/`

Qwen API Key:

1. Go to `https://dashscope.console.aliyun.com/`
2. Sign up or log in
3. Create an API key
4. Copy the key

3. Configure Environment

```
cp .env.example .env
```

Edit `.env` and fill in your credentials:

```
MIRO_ACCESS_TOKEN=your_miro_access_token_here  
MIRO_BOARD_ID=your_board_id_here  
QWEN_API_KEY=your_qwen_api_key_here  
QWEN_API_URL=https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions  
QWEN_MODEL=qwen-plus  
PORT=3000
```

4. Add Sticky Notes to Your Miro Board

Add at least 2 sticky notes with random topics. For best results, use 6-8 unrelated ideas like:

- Pokémon
- Hyderabad
- Rain
- Coffee
- AI
- Cricket
- Traffic
- College

5. Run the App

```
npm start
```

Open `http://localhost:3000`

6. Click "Connect the Impossible"

Watch as AI finds surprising connections and creates new idea cards on your Miro board.

7. Click "Make It Weirder"

Push the ideas even further.

How It Works

1. **Reads sticky notes** from your Miro board via the Miro REST API
2. **Sends them to Qwen** (qwen-plus model) with a creative ideation prompt
3. **Qwen finds surprising connections** and generates 3 ideas:
 - One practical
 - One unusual
 - One borderline ridiculous but logically defensible
4. **Ideas are written back to the board** as visually distinct cards with connectors
5. **"Make It Weirder"** pushes the ideas further with amplified creativity

Demo in 90 Seconds

1. Pre-populate a board with: Pokémon, Hyderabad, Rain, Coffee, AI, Cricket, Traffic, College
2. Run `npm start`
3. Open `http://localhost:3000`
4. Click "Connect the Impossible"
5. Watch ideas appear on the board
6. Click "Make It Weirder"
7. Show the board — it IS the output

Project Structure

```
sticky-note-roulette/  
├─ package.json           # Dependencies  
├─ .env.example           # Environment template  
├─ server.js              # Express server with Miro + Qwen integration  
├─ public/  
│   ├─ index.html        # Frontend UI  
│   ├─ style.css          # Dark, playful styling  
│   └─ app.js             # Frontend logic  
└─ README.md              # This file
```

Tech Stack

- **Backend:** Node.js + Express
- **Miro Integration:** @mirohq/miro-api + REST API
- **AI:** Qwen (qwen-plus) via OpenAI-compatible API
- **Frontend:** Vanilla JS, no frameworks

Troubleshooting

No sticky notes found:

- Make sure you've added sticky notes to your Miro board
- Check that your `MIRO_BOARD_ID` is correct

Miro API errors:

- Verify your `MIRO_ACCESS_TOKEN` has `boards:read` and `boards:write` scopes
- Check that the token hasn't expired

Qwen API errors:

- Verify your `QWEN_API_KEY` is valid
- Check your Alibaba Cloud account has credits

Ideas not appearing on board:

- Refresh your Miro board
- Check the server console for errors

Extending the Prototype

Ideas for next steps:

- Add support for images/shapes, not just sticky notes
- Let users select which sticky notes to connect
- Add different AI "personalities" (practical, absurd, poetic)
- Generate visual diagrams, not just text cards
- Multi-board support
- Collaborative sessions with multiple users

License

MIT — Do whatever you want with this.

Built with Miro + Qwen + coffee ☕



Installation Steps

1. Create a new directory and save all files
2. Run `npm install`
3. Copy `.env.example` to `.env` and fill in your credentials
4. Add 6-8 random sticky notes to your Miro board
5. Run `node server.js`
6. Open `http://localhost:3000`
7. Click "Connect the Impossible"

All code is complete and ready to run. No placeholders, no abbreviations.